

LAB 2 — CONTAINERIZATION & KUBERNETES ORCHESTRATION

Task 2 — Manage Docker Containers and Images

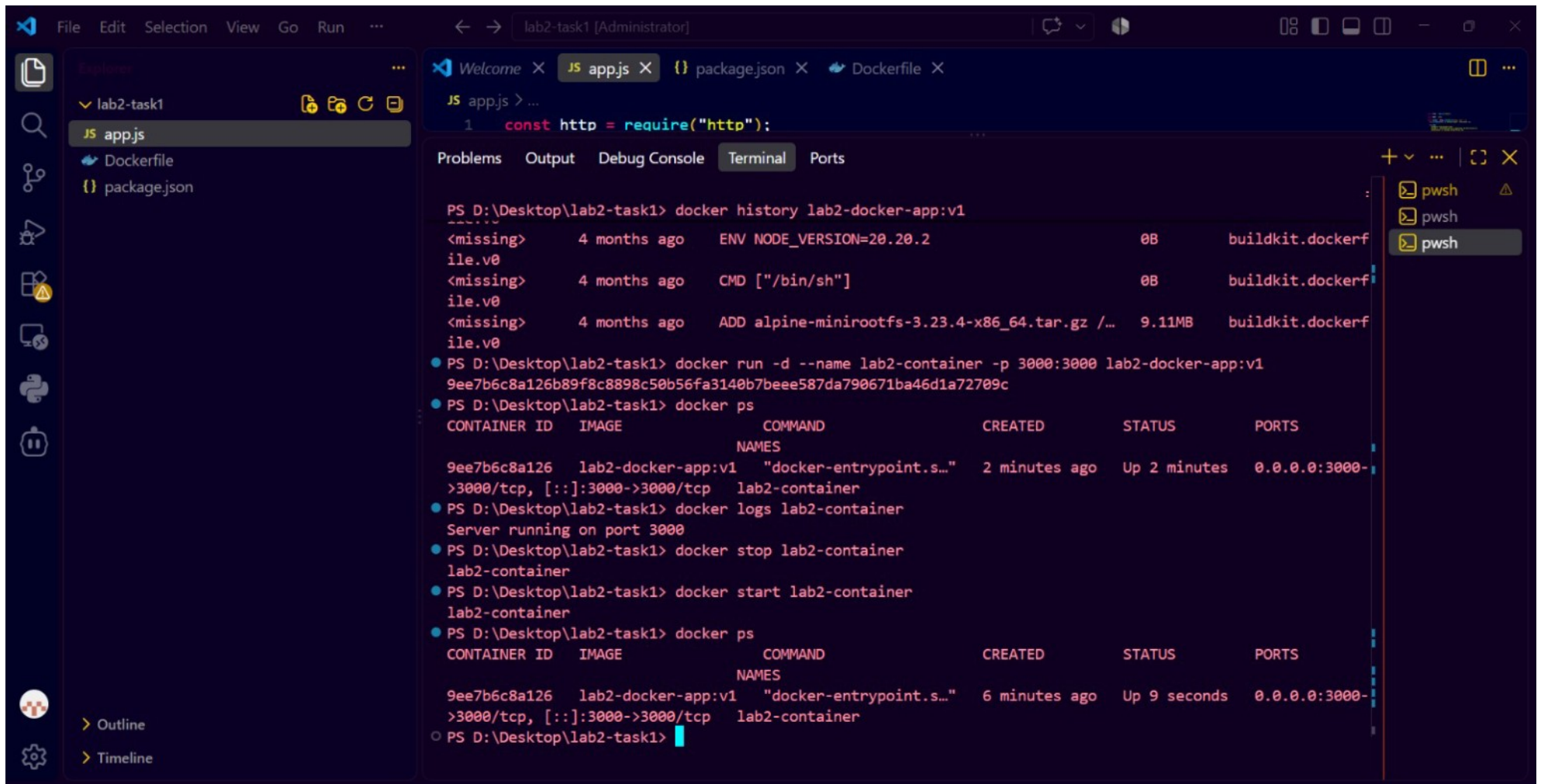
Container lifecycle • Image tagging • Registry push • Docker Compose

Student	Kunal Prajapati
Local image	lab2-docker-app:v1
Docker Hub image	kunalprajapati1/lab2-docker-app:v1
Compose services	web (lab2-docker-app:v1) + redis (redis:7-alpine)

This HD revision uses the exact screenshots supplied in the latest message. Screenshots are embedded at their native resolution and displayed on landscape pages to avoid the small/blurred appearance of the earlier report.

1. Container Lifecycle Management

The evidence shows **lab2-container** being run, inspected with **docker ps**, logged, stopped, and then removed. The final **docker ps -a** output confirms that the removed container no longer appears.



The screenshot shows a Visual Studio Code editor with a terminal window open. The terminal displays the following commands and output:

```
PS D:\Desktop\lab2-task1> docker history lab2-docker-app:v1
<missing>      4 months ago      ENV  NODE_VERSION=20.20.2                0B      buildkit.dockerf
ile.v0
<missing>      4 months ago      CMD  ["/bin/sh"]                          0B      buildkit.dockerf
ile.v0
<missing>      4 months ago      ADD  alpine-minirootfs-3.23.4-x86_64.tar.gz /... 9.11MB  buildkit.dockerf
ile.v0
• PS D:\Desktop\lab2-task1> docker run -d --name lab2-container -p 3000:3000 lab2-docker-app:v1
9ee7b6c8a126b89f8c8898c50b56fa3140b7beee587da790671ba46d1a72709c
• PS D:\Desktop\lab2-task1> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
9ee7b6c8a126   lab2-docker-app:v1                 "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  lab2-container
• PS D:\Desktop\lab2-task1> docker logs lab2-container
Server running on port 3000
• PS D:\Desktop\lab2-task1> docker stop lab2-container
lab2-container
• PS D:\Desktop\lab2-task1> docker start lab2-container
lab2-container
• PS D:\Desktop\lab2-task1> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
9ee7b6c8a126   lab2-docker-app:v1                 "docker-entrypoint.s..." 6 minutes ago  Up 9 seconds  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  lab2-container
• PS D:\Desktop\lab2-task1>
```

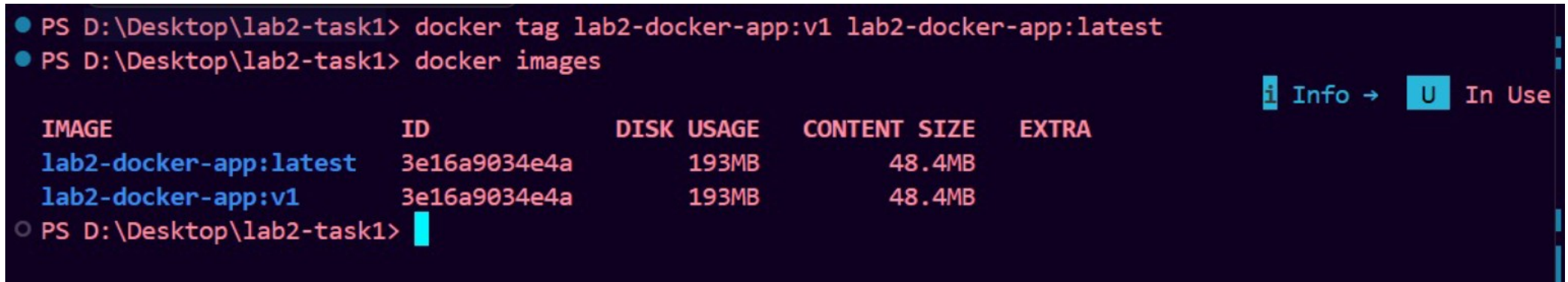
Figure 1. Original supplied screenshot: docker run, docker ps, docker logs, docker stop, and docker start evidence. The screenshot is presented without artificial blur or replacement content.

```
● PS D:\Desktop\lab2-task1> docker stop lab2-container
lab2-container
● PS D:\Desktop\lab2-task1> docker rm lab2-container
lab2-container
● PS D:\Desktop\lab2-task1> docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
○ PS D:\Desktop\lab2-task1> 
```

Figure 2. Original supplied screenshot: docker stop, docker rm, and final docker ps -a showing no containers.

2. Docker Image Tagging

The image **lab2-docker-app:v1** was tagged as **lab2-docker-app:latest**. The supplied image listing shows both tags using the same image ID **3e16a9034e4a**.



A terminal window with a dark background and light blue text. The prompt is 'PS D:\Desktop\lab2-task1>'. The first command is 'docker tag lab2-docker-app:v1 lab2-docker-app:latest'. The second command is 'docker images'. Below the command is a table of Docker images. The table has five columns: IMAGE, ID, DISK USAGE, CONTENT SIZE, and EXTRA. There are two rows of data. The first row shows 'lab2-docker-app:latest' with ID '3e16a9034e4a', DISK USAGE '193MB', and CONTENT SIZE '48.4MB'. The second row shows 'lab2-docker-app:v1' with the same ID, DISK USAGE, and CONTENT SIZE. The prompt is followed by a cursor. On the right side of the terminal, there are two buttons: 'Info' and 'In Use'.

```
PS D:\Desktop\lab2-task1> docker tag lab2-docker-app:v1 lab2-docker-app:latest
PS D:\Desktop\lab2-task1> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
lab2-docker-app:latest	3e16a9034e4a	193MB	48.4MB	
lab2-docker-app:v1	3e16a9034e4a	193MB	48.4MB	

```
PS D:\Desktop\lab2-task1>
```

Figure 3. Original supplied screenshot: docker tag lab2-docker-app:v1 lab2-docker-app:latest followed by docker images.

3. Docker Registry Login and Registry Tag

Docker login succeeded for the Docker Hub account **kunalprajapati1**. The image was then tagged as **kunalprajapati1/lab2-docker-app:v1**; the image listing confirms the registry-qualified tag.

```
PS D:\Desktop\lab2-task1> docker login
Authenticating with existing credentials... [Username: kunalprajapati1]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
PS D:\Desktop\lab2-task1> docker tag lab2-docker-app:v1 kunalprajapati1/lab2-docker-app:v1
PS D:\Desktop\lab2-task1> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
kunalprajapati1/lab2-docker-app:v1	3e16a9034e4a	193MB	48.4MB	
lab2-docker-app:latest	3e16a9034e4a	193MB	48.4MB	
lab2-docker-app:v1	3e16a9034e4a	193MB	48.4MB	

```
PS D:\Desktop\lab2-task1>
```

Figure 4. Original supplied screenshot: successful Docker login, registry-qualified docker tag command, and docker images output.

4. Push Image to Docker Registry

The registry-qualified image was pushed using **docker push kunalprajapati1/lab2-docker-app:v1**. All displayed layers report **Pushed**, followed by the v1 digest.

```
PS D:\Desktop\lab2-task1> docker push kunalprajapati1/lab2-docker-app:v1
The push refers to repository [docker.io/kunalprajapati1/lab2-docker-app]
f8da4a911a58: Pushed
44136fa355b3: Pushed
4feea04c1543: Pushed
6a0ac1617861: Pushed
b2cbbfe903b0: Pushed
fff4e2c1b189: Pushed
a4844149f63e: Pushed
e44968c2c641: Pushed
82027f1f4aaa: Pushed
v1: digest: sha256:3e16a9034e4a296dc57d10e09436126eed31e801a24c09bf8da763e00ab34d5e size: 856
PS D:\Desktop\lab2-task1>
```

Figure 5. Original supplied screenshot: successful Docker Hub image push with returned SHA256 digest.

5. Docker Compose — Multi-Container Setup

Docker Compose was used to start the web application and Redis. The supplied evidence explicitly shows the Redis image being pulled and both **lab2-compose-web** and **lab2-compose-redis** being started.

```
✓ Network lab2-task1_default Removed 0.6s
● PS D:\Desktop\lab2-task1> docker compose up -d
[+] up 13/13
✓ Image redis:7-alpine Pulled 12.0s
✓ Network lab2-task1_default Created 0.1s
✓ Container lab2-compose-web Started 1.1s
✓ Container lab2-compose-redis Started 0.7s
◎ PS D:\Desktop\lab2-task1> docker compose ps
```

Figure 6. Original supplied screenshot: docker compose up -d showing redis:7-alpine pulled and both Compose containers started.

An additional supplied run shows the web service starting and **docker compose ps** reporting the web service with port 3000 published.

```
● PS D:\Desktop\lab2-task1> docker compose up -d
[+] up 2/2
✓ Network lab2-task1_default Created 0.0s
✓ Container lab2-compose-web Started 0.6s
● PS D:\Desktop\lab2-task1> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
lab2-compose-web    lab2-docker-app:v1  "docker-entrypoint.s..."  web        6 seconds ago  Up 5 seconds  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
○ PS D:\Desktop\lab2-task1>
```

Figure 7. Original supplied screenshot: web service startup and docker compose ps output.

6. Docker Compose Service Verification

The supplied **docker compose ps** output confirms both services are running: **lab2-compose-redis** uses **redis:7-alpine**, while **lab2-compose-web** uses **lab2-docker-app:v1** and publishes port 3000.

```
PS D:\Desktop\lab2-task1> docker compose ps
NAME                IMAGE                COMMAND              SERVICE    CREATED        STATUS          PORTS
lab2-compose-redis  redis:7-alpine      "docker-entrypoint.s...  redis      42 seconds ago Up 41 seconds   6379/
tcp
lab2-compose-web    lab2-docker-app:v1  "docker-entrypoint.s...  web        42 seconds ago Up 40 seconds   0.0.0
.0:3000->3000/tcp, [::]:3000->3000/tcp
PS D:\Desktop\lab2-task1>
```

Figure 8. Original supplied screenshot: docker compose ps showing the web and Redis services with their status and ports.

7. Task 2 Requirement Checklist

Assessment requirement	Direct screenshot evidence	Status
List / start / stop / remove containers	Figures 1–2: docker ps, docker start, docker stop, docker rm, docker ps -a.	Completed
Tag image	Figure 3: v1 → latest tag and docker images.	Completed
Push image to registry	Figures 4–5: Docker login, registry tag, successful docker push and digest.	Completed
Docker Compose multi-container setup	Figures 6–8: Compose startup and docker compose ps showing web + Redis.	Completed

Conclusion: The supplied evidence demonstrates the required Task 2 operations: container lifecycle management, image tagging, registry authentication and push, and a multi-container Docker Compose setup.

Important: No screenshot has been invented, blurred, or substituted. The latest supplied screenshots are the evidence used in this report.